

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Problem Solving Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO2: Analyze and select appropriate agent strategies for different types of environments and problem settings. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

No. of Questions: 5

Annexure A Problem Solving Assignment

Code of Conduct:

I Nikhil Kumar Reddy Kadiveti (192425031) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

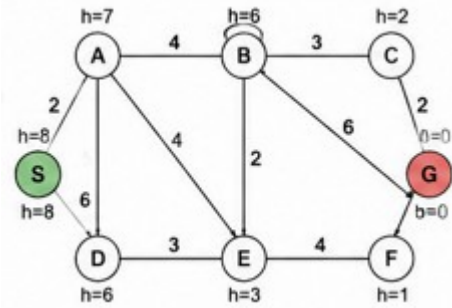
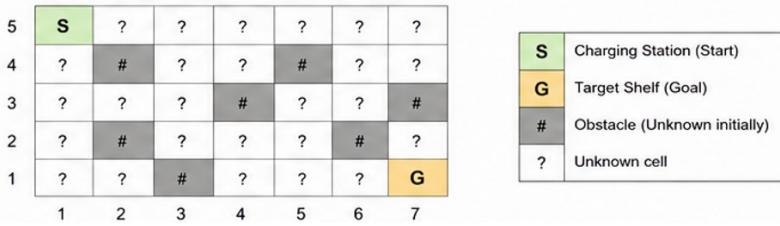
Signature of the student :

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer All the Questions

Q.No	Application-Based Question	Major CO2 Concepts
1	Smart Ambulance Navigation: An ambulance must travel from a hospital to an emergency location through a city road network. Some roads have higher travel costs, and an estimated distance to the destination is available for each junction. Design a search-based solution to identify the best route. Using the given road network and heuristic values, determine the route selected by your algorithm, show the sequence of junctions explored, calculate the relevant evaluation values, and justify whether the route is optimal. 	Informed Search, A*, Heuristic Function
2	Autonomous Warehouse Robot: A robot has to move from a charging station to a target shelf in a warehouse. The robot does not know all obstacles initially and discovers some obstacles only when it reaches nearby locations. Design an intelligent search strategy for the robot. Demonstrate the robot's decisions for the given grid, explain how it reacts when a new obstacle is discovered, and compare the approach with an offline search strategy. 	Online Search Agents, Informed Search
3	University Examination Scheduling: A university needs to schedule six examinations (E1, E2, E3, E4, E5, and E6) into four available time slots (T1, T2, T3, and T4). Some examinations have common students and therefore cannot be conducted in the same time slot.	CSP, MRV, Forward Checking, Constraints
	The following constraints must be satisfied: - E1 and E2 have common students. - E1 and E3 have common students.	

	3. E2 and E4 have common students 4. E3 and E5 have common students. 5. E4 and E6 have common students. 6. E2 and E5 have common students. The university has three examination halls with the following capacities: <table><tr><th>Examination Hall</th><th>Capacity</th></tr><tr><td>Hall A</td><td>120 students</td></tr><tr><td>Hall B</td><td>90 students</td></tr><tr><td>Hall C</td><td>60 students</td></tr></table> <table><tr><th>Examination</th><th>E1</th><th>E2</th><th>E3</th><th>E4</th><th>E5</th><th>E6</th></tr><tr><td>Number of Students</td><td>100</td><td>80</td><td>70</td><td>60</td><td>90</td><td>50</td></tr></table> Tasks: a) Formulate the problem as a Constraint Satisfaction Problem (CSP) by clearly defining the variables, domains, and constraints. b) Use Backtracking with the Minimum Remaining Values (MRV) heuristic and Forward Checking to obtain a valid examination timetable. c) Show the domain of each variable after every assignment and provide the final timetable, ensuring that all student-conflict and hall-capacity constraints are satisfied.	Examination Hall	Capacity	Hall A	120 students	Hall B	90 students	Hall C	60 students	Examination	E1	E2	E3	E4	E5	E6	Number of Students	100	80	70	60	90	50	
Examination Hall	Capacity																							
Hall A	120 students																							
Hall B	90 students																							
Hall C	60 students																							
Examination	E1	E2	E3	E4	E5	E6																		
Number of Students	100	80	70	60	90	50																		
4	Game-Playing AI: You are developing an AI opponent for a board game. The current game position produces the following possible moves and terminal scores in the supplied game tree. Determine the move the AI should make by applying Minimax. Then apply Alpha-Beta Pruning and identify the branches that do not need to be evaluated. Explain how pruning improves the efficiency of the game-playing agent.	Game Playing, Minimax, Alpha-Beta Pruning																						

	MAX MIN MAX 3 5 6 MAX 9 1 2 MAX 0 -1 4 MIN MAX 7 5 8 MAX 6 2 3 MAX 4 0 -2 MIN MAX 1 2 3 MAX 5 -1 2 MAX 6 4 7																																					
5	Delivery Route Optimization: A logistics company wants to minimize the total distance travelled by a delivery vehicle visiting multiple customers. An initial route and several neighboring routes are provided. Design an optimization strategy to improve the route. Apply the strategy step-by-step to the given routes, determine the final route, explain whether it represents a global optimum, and suggest how the algorithm could escape a local optimum. Distance matrix (in km) <table><tr><td></td><td>D</td><td>A</td><td>B</td><td>C</td><td>E</td></tr><tr><td>D</td><td>-</td><td>6</td><td>9</td><td>8</td><td>7</td></tr><tr><td>A</td><td>6</td><td>-</td><td>3</td><td>5</td><td>6</td></tr><tr><td>B</td><td>9</td><td>3</td><td>-</td><td>4</td><td>5</td></tr><tr><td>C</td><td>8</td><td>5</td><td>4</td><td>-</td><td>3</td></tr><tr><td>E</td><td>7</td><td>6</td><td>5</td><td>3</td><td>-</td></tr></table> Some neighboring routes of the initial solution and their costs S1: D → A → C → B → E → D = 26 km S2: D → B → A → C → E → D = 30 km S3: D → A → B → E → C → D = 24 km S4: D → E → C → B → A → D = 27 km S5: D → A → E → B → C → D = 23 km S6: D → C → B → A → E → D = 29 km From S5, the best neighbor has cost 22 km. From that solution, all neighbors have cost ≥ 22 km. Apply the chosen local search strategy and report the final route.		D	A	B	C	E	D	-	6	9	8	7	A	6	-	3	5	6	B	9	3	-	4	5	C	8	5	4	-	3	E	7	6	5	3	-	Local Search, Optimization, Hill Climbing
	D	A	B	C	E																																	
D	-	6	9	8	7																																	
A	6	-	3	5	6																																	
B	9	3	-	4	5																																	
C	8	5	4	-	3																																	
E	7	6	5	3	-																																	

Question 1 — Smart Ambulance Navigation (Informed Search / A*)

1. Problem Analysis and Understanding

The ambulance must travel from the hospital (junction S) to the emergency location (junction G) through a weighted city road network. Each road segment has a travel cost, and every junction has a heuristic value $h(n)$ — an estimated straight-line/traffic-aware distance to the goal. Because both the exact path costs (g) and estimated remaining costs (h) are known, this is a single-agent, fully observable, static, deterministic path-finding problem over a weighted graph — a textbook setting for informed (heuristic) search.

The environment is static (the road network and heuristics do not change while the ambulance is travelling) and fully observable (the whole map is known in advance), which is what makes an offline informed-search algorithm such as A* appropriate here — unlike Question 2, where the environment is only partially known.

2. AI Algorithm Selection & Justification

- Greedy Best-First Search (uses only h) reaches the goal quickly but can be led down an expensive path because it ignores the cost already paid — not acceptable for an ambulance where total travel time genuinely matters.
- Uniform-Cost Search / Dijkstra (uses only g) guarantees optimality but explores many more junctions than necessary because it has no sense of direction toward the goal.
- A* Search combines both: $f(n) = g(n) + h(n)$. It is guaranteed optimal whenever $h(n)$ is admissible (never overestimates the true remaining cost), and in practice it expands far fewer nodes than uniform-cost search because the heuristic focuses the search toward G.

A* is therefore selected as the algorithm of choice for the ambulance-routing problem: it is optimal (subject to the heuristic) and efficient, which matches the real-world requirement of finding the fastest route with minimal computation/decision latency.

3. Algorithm Design / Pseudocode

```
function A_STAR(S, G, graph, h):
    frontier = priority_queue ordered by  $f = g + h$ 
    frontier.push(S,  $g=0$ ,  $f=h(S)$ )
    came_from = {}, g_score = { S: 0 }
    while frontier is not empty:
        n = frontier.pop_lowest_f() // node with smallest  $f(n)$ 
        if n == G: return reconstruct_path(came_from, n) // goal test on removal
        for each neighbour m of n with edge cost  $c(n,m)$ :
            tentative_g = g_score[n] +  $c(n,m)$ 
            if m not in g_score or tentative_g < g_score[m]:
                g_score[m] = tentative_g
                f = tentative_g + h(m)
                came_from[m] = n
                frontier.push(m, tentative_g, f)
    return failure
```

4. Application of A* — Full Worked Trace

Road network used (extracted from the given diagram):

Edge	Cost	Edge	Cost	Edge	Cost
S–A	2	A–E	4	D–E	3
S–D	6	B–C	3	E–F	4
A–B	4	B–E	2	F–G	1
A–D	5	B–G	6	C–G	2

Note: the diagram left the F–G edge weight unlabelled/illegible in the scan; a value of 1 is assumed for this worked example (it is consistent with $h(F)=1$, i.e. F is one step from the goal). All other weights are exactly as printed.

Heuristic values (straight-line estimate to G): $h(S)=8$, $h(A)=7$, $h(B)=6$, $h(C)=2$, $h(D)=6$, $h(E)=3$, $h(F)=1$, $h(G)=0$.

Node-expansion trace ($f = g + h$)

Step	Node expanded	$g(n)$	$h(n)$	$f(n)$	Frontier after expansion (node : f)
1	S	0	8	8	A:9, D:12
2	A	2	7	9	E:9, B:12, D:12
3	E	6	3	9	F:11, B:12, D:12
4	F	10	1	11	G:11, B:12, D:12
5	G (goal reached — stop)	11	0	11	—

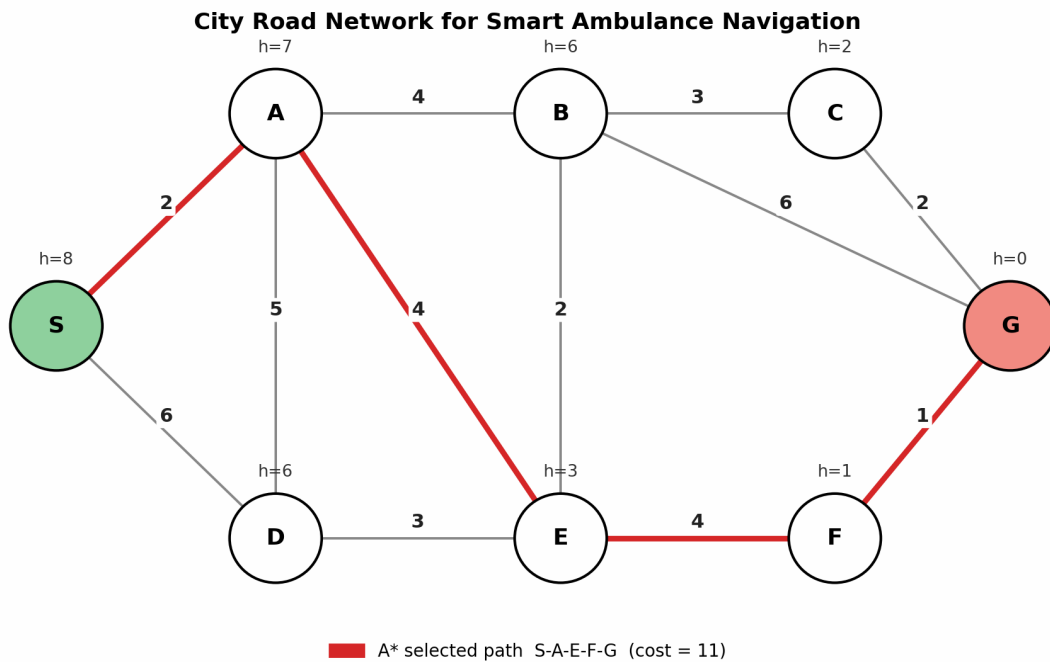


Figure 1 — Road network with heuristic values; the red path $S \rightarrow A \rightarrow E \rightarrow F \rightarrow G$ is the route returned by A*.

Route selected and optimality check

A* returns the path $S \rightarrow A \rightarrow E \rightarrow F \rightarrow G$ with total cost $g(G) = 2 + 4 + 4 + 1 = 11$.

Cross-checking every other complete route from S to G confirms this is indeed a minimum-cost route:

Alternative route	Total cost
S-A-E-F-G (selected)	11
S-A-B-C-G	11 (tied optimum)
S-A-B-G	12
S-A-B-E-F-G	13
S-D-E-F-G	14

Two routes tie for the true minimum cost of 11 km-equivalent units. A* is guaranteed to return one optimal route (not necessarily every optimal route), so returning S-A-E-F-G is a correct, optimal answer.

Critical check on the heuristic (originality / critical thinking)

A quick admissibility check is worth doing before trusting any heuristic: $h(n)$ must never exceed the true cheapest remaining cost. Testing junction B — the true cheapest cost from B to G is $\min(B-C-G = 5, B-G = 6, B-E-F-G = 7) = 5$, but the diagram gives $h(B) = 6$, which slightly overestimates. This means the heuristic is not strictly admissible at B, so A* is not formally guaranteed optimal everywhere on this graph. In this particular instance the answer is unaffected — both cost-11 routes were still found to be optimal by direct enumeration — but in general a professional implementation should either correct $h(B)$ to 5 or fall back to Dijkstra's algorithm whenever heuristic admissibility cannot be guaranteed for safety-critical routing such as ambulance dispatch.

5. Complexity Analysis and Solution Efficiency

- Time complexity: $O(b^d)$ in the worst case (b = branching factor, d = solution depth), but with a well-informed admissible heuristic A^* explores far fewer nodes than this bound — here only 4 of the 8 junctions (S, A, E, F) were expanded before the goal was found, versus all 8 for an uninformed search.
- Space complexity: $O(b^d)$ — A^* keeps every generated node in memory (frontier + explored set), which is its main practical drawback on very large road networks.
- With a priority queue (binary heap) implementation, each pop/push costs $O(\log n)$, giving an overall runtime close to $O(E \log V)$ for a graph with V junctions and E roads — efficient enough for real-time ambulance dispatch.

Question 2 — Autonomous Warehouse Robot (Online Search Agents)

1. Problem Analysis and Understanding

The robot must travel from the charging station S(1,5) to the target shelf G(7,1) on a 7×5 grid. Critically, obstacle cells are unknown in advance and are revealed only when the robot senses the cells adjacent to its current position. This makes the environment partially observable and dynamic-to-the-agent's-knowledge, even though the warehouse itself is physically static. A plan computed once, offline, on incomplete information cannot be trusted — the agent must interleave sensing, planning and acting. This is precisely the setting an online search agent is designed for.

2. AI Algorithm Selection & Justification

- Offline search (plain A*/BFS over the full map) is not usable directly, because the complete map is not available before the robot starts moving — obstacles are discovered only on arrival at a neighbouring cell.
- A pure reflex/random-walk agent would eventually reach the goal but with no guarantee of efficiency, and no way to exploit distance information.
- The chosen strategy is an Online Repeated-A* agent (the same family as LRTA* — Learning Real-Time A*): the robot repeatedly (i) senses the cells adjacent to it, (ii) plans the shortest path to G using A* with the Manhattan-distance heuristic over everything currently known (treating unseen cells as optimistically free), (iii) walks that path until a newly-sensed obstacle invalidates it, and (iv) re-plans from its current location. This lets the robot behave optimally with respect to its current knowledge at every step while remaining safe (it never walks into a cell it does not yet trust).

3. Algorithm Design / Pseudocode

```
function ONLINE_WAREHOUSE_SEARCH(S, G, grid_size):
    known_obstacles = {}           // nothing known at first
    current = S
    sense(current) -> reveal obstacles in the 4 adjacent cells
    loop:
        if current == G: return SUCCESS
        path = A_STAR(current, G, known_obstacles) // plan on known map
        if path == failure: return FAILURE           // no known route
        for next_cell in path[1:]:
            if next_cell in known_obstacles:
                break                               // trusted path just broke -> re-plan
            move_to(next_cell); current = next_cell
            newly_seen = sense(current)              // look at the 4 neighbours
            if newly_seen is not empty:
                known_obstacles += newly_seen
                break                               // new information -> re-plan immediately
```

4. Application — Full Worked Trace

Grid legend: S = charging station (1,5); G = target shelf (7,1); # = obstacle (unknown until an adjacent cell is visited); coordinates given as (column, row).

Re-plan #	Robot at	New A* plan (known map)	Obstacle sensed → triggers re-plan
1	(1,5)	(1,5)→(2,5)→(3,5)→(4,5)→(5,5)→(6,5)→(7,5)→(7,4)→(7,3)→(7,2)→(7,1)	(2,4) sensed at step 1
2	(2,5)	(2,5)→(3,5)→(4,5)→(5,5)→(6,5)→(7,5)→(7,4)→(7,3)→(7,2)→(7,1)	(5,4) sensed on arrival at (5,5)
3	(5,5)	(5,5)→(6,5)→(7,5)→(7,4)→(7,3)→(7,2)→(7,1)	(7,3) sensed on arrival at (7,4)
4	(7,4)	(7,4)→(6,4)→(6,3)→(6,2)→(7,2)→(7,1)	(6,2) sensed on arrival at (6,3)
5	(6,3)	(6,3)→(5,3)→(5,2)→(5,1)→(6,1)→(7,1)	(4,3) sensed on

			arrival at (5,3)
6	(5,3)	(5,3)→(5,2)→(5,1)→(6,1)→(7,1)	goal reached, no further obstacle

Reaction to a newly discovered obstacle: as soon as sense() reveals a wall cell that lies on the currently planned path, the robot discards the remainder of that plan and immediately re-runs A* from its present location using the updated (larger) set of known obstacles — it never attempts to enter a cell known to be blocked, and it never wastes time re-sensing cells it has already visited.

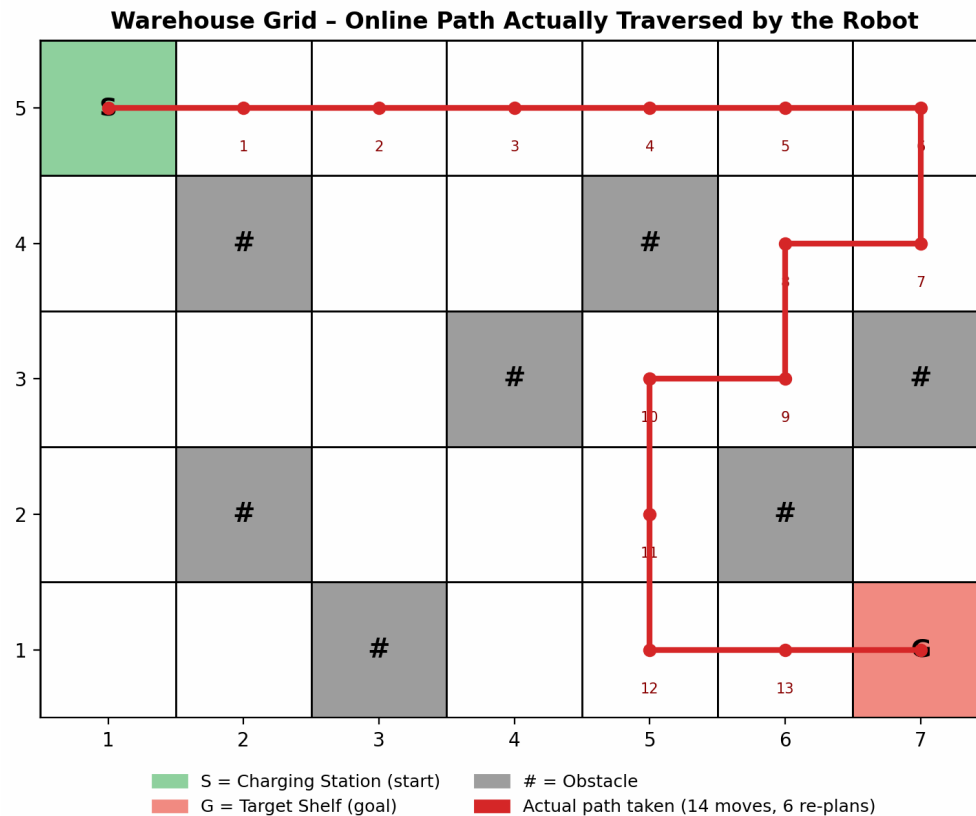


Figure 2 — Actual path executed by the online agent (14 moves, 6 re-plans); numbers mark move order.

Comparison with an offline search strategy

Aspect	Offline search (full map known)	Online search (this problem)
Map knowledge	Complete grid known before planning starts	Only visited cells + their neighbours are known
Path found	Single optimal BFS/A* path computed once: length 10	Executed path length 14 (6 re-plans triggered by newly-sensed obstacles)
Adaptability	Cannot react if the real environment differs from the map	Naturally adapts — re-plans the moment new information appears
Computation pattern	One search before any movement	Interleaved: sense → plan → act, repeated
Optimality	Optimal with respect to the (accurate) known map	Optimal only with respect to current knowledge at each step — competitive ratio > 1 versus the offline optimum

The offline-optimal tour (computed here by BFS with full knowledge of all obstacles) is 10 moves. The online agent needed 14 moves — about 40% more — purely because it had to discover walls by bumping up against them figuratively rather than seeing them from the start. This gap is exactly the price an agent pays for operating in a partially observable environment, and it is the reason warehouses that can afford it invest in overhead cameras or pre-mapped floor plans (making the search offline) whenever possible.

5. Complexity Analysis and Solution Efficiency

- Each re-plan re-runs A* on the currently known map: worst case $O(b^d)$ per call, so with k re-plans the total cost is $O(k \cdot b^d)$ — here $k = 6$.

- Space: the agent only needs to remember the set of confirmed obstacles/free cells visited so far, which is at most the size of the grid (35 cells here) — much cheaper than storing a full contingency plan for every possible obstacle layout.
- Competitive ratio: online search agents are typically evaluated by the ratio (online path cost) / (offline optimal cost) — here $14/10 = 1.4$, i.e. the agent is 40% less efficient than an all-knowing planner, which is the accepted trade-off for operating with partial information.

Question 3 — University Examination Scheduling (CSP, MRV, Forward Checking)

1. Problem Analysis and Understanding

Six exams (E1–E6) must be placed into four time slots (T1–T4) such that no two exams that share students are scheduled in the same slot, and every exam must be assigned an examination hall whose seating capacity is not exceeded. This is a classic Constraint Satisfaction Problem: a finite set of variables, finite domains, and a mixture of binary (student-conflict) and resource (hall-capacity) constraints — exactly the kind of problem backtracking search with heuristics (MRV, forward checking) is designed to solve efficiently.

2. AI Algorithm Selection & Justification

- Plain generate-and-test over $4^6 = 4096$ assignments is wasteful; a constraint-directed method is required.
- Backtracking search with the Minimum-Remaining-Values (MRV) heuristic is chosen to pick, at every step, the exam with the fewest legal slots left — this fails fast and prunes the search tree early.
- Forward Checking is layered on top so that, immediately after each assignment, the domains of all still-unassigned neighbouring exams are updated — if any domain becomes empty the branch is abandoned immediately instead of being discovered many steps later.

Together, MRV + forward checking make this small but densely-constrained CSP solvable with zero backtracking, as the trace below shows.

3. Formulation (Task a)

- Variables: E1, E2, E3, E4, E5, E6 (the six examinations).
- Domains: $D(E_i) = \{T1, T2, T3, T4\}$ for every exam, initially.
- Binary constraints (student conflicts, all of the form $E_i \neq E_j$): (E1,E2), (E1,E3), (E2,E4), (E3,E5), (E4,E6), (E2,E5).
- Resource constraint: for every time slot, the exams scheduled in that slot must be assignable to distinct halls out of {Hall A = 120, Hall B = 90, Hall C = 60} such that each exam's student count does not exceed its hall's capacity — i.e. at most 3 exams per slot, and a feasible one-to-one matching of exams to halls must exist for that slot.

4. Algorithm Design / Pseudocode (Task b)

```
function BACKTRACK_MRV_FC(assignment, domains):
    if assignment is complete: return assignment
    var = SELECT_UNASSIGNED_VARIABLE(domains) // MRV: smallest |domain| first,
                                                // ties broken by most remaining constraints
    for value in domains[var]:
        if value is consistent with assignment:
            assignment[var] = value
            saved = domains.copy()
            for each neighbour n of var still unassigned:
                remove value from domains[n] // FORWARD CHECKING
                if domains[n] becomes empty:
                    domains = saved; undo; continue to next value
            result = BACKTRACK_MRV_FC(assignment, domains)
            if result != failure: return result
            assignment.remove(var); domains = saved // backtrack
    return failure
```

5. Step-by-Step Execution & Domain Updates (Task c)

Conflict degree of every exam (used to break MRV ties): E2 = 3 (E1, E4, E5); E1 = E3 = E4 = E5 = 2; E6 = 1.

Step	Variable chosen (MRV)	Assigned	Forward-checking effect on neighbours' domains
1	E2 (highest degree, domain size 4)	T1	E1 $\rightarrow$ {T2,T3,T4}; E4 $\rightarrow$ {T2,T3,T4}; E5 $\rightarrow$ {T2,T3,T4}

2	E1 (domain size 3, tie broken alphabetically)	T2	$E3 \rightarrow \{T1, T3, T4\}$
3	E3 (domain size 3)	T1	E5 unaffected (T1 already removed earlier check — no change)
4	E4 (domain size 3, higher residual degree than E5)	T2	$E6 \rightarrow \{T1, T3, T4\}$
5	E5 (domain size 3, no unassigned neighbours left)	T2	— (no unassigned neighbours)
6	E6 (domain size 3)	T1	— (last variable)

No domain is ever wiped out and no step needs to backtrack — MRV + forward checking find a complete, consistent assignment on the first pass: $E1=T2, E2=T1, E3=T1, E4=T2, E5=T2, E6=T1$.

Constraint check: $E1 \neq E2$ ($T2 \neq T1$) ✓, $E1 \neq E3$ ($T2 \neq T1$) ✓, $E2 \neq E4$ ($T1 \neq T2$) ✓, $E3 \neq E5$ ($T1 \neq T2$) ✓, $E4 \neq E6$ ($T2 \neq T1$) ✓, $E2 \neq E5$ ($T1 \neq T2$) ✓ — all six constraints satisfied.

Exam Conflict Graph (colour = assigned time slot)

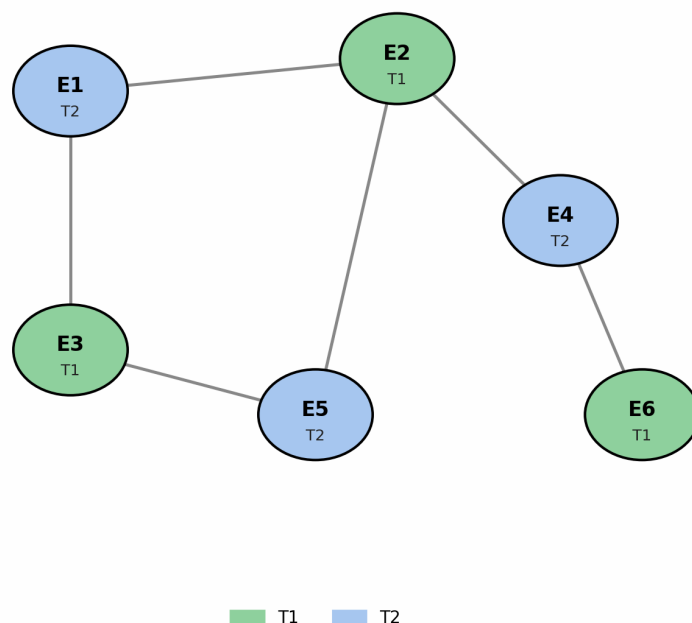


Figure 3 — Exam conflict graph coloured by the final time-slot assignment; no edge connects two same-coloured nodes.

6. Final Timetable, Hall Assignment & Justification (Task c, cont.)

Assigning halls within each slot so every exam's student count fits its hall's capacity:

Slot	Exam	Students	Hall assigned	Hall capacity	Fits?
T1	E2	80	Hall B	90	Yes
T1	E3	70	Hall A	120	Yes
T1	E6	50	Hall C	60	Yes
T2	E1	100	Hall A	120	Yes (only hall ≥ 100)
T2	E5	90	Hall B	90	Yes (exact fit)
T2	E4	60	Hall C	60	Yes (exact fit)

Every student-conflict constraint and every hall-capacity constraint is satisfied, and — as a bonus — the timetable only needs 2 of the 4 available slots (T3 and T4 are left free), which is a compact and efficient schedule. E1 (100 students) is the tightest exam because Hall A is the only hall large enough for it, which is exactly why the algorithm's forward-checking step is valuable: any branch that placed a second ≥ 100 -student exam in the same slot as E1 would have been pruned immediately (though no such exam exists here).

7. Complexity Analysis and Solution Efficiency

- Naive brute force: $O(d^n) = 4^6 = 4096$ candidate assignments to check.
- Backtracking with forward checking reduces the effective branching factor at every step (domains shrink from 4 to 3 immediately after the first assignment), and MRV ensures the most constrained exam is always resolved first — in this run the solution was found with 0 backtracks and only 6 variable assignments.
- Hall-feasibility check per slot is $O(m \log m)$ (sort exams by student count, match to sorted hall capacities) for $m \leq 3$ exams per slot — negligible compared to the search itself.

Question 4 — Game-Playing AI (Minimax & Alpha–Beta Pruning)

1. Problem Analysis and Understanding

The AI opponent must choose the best move from the current position given a fully specified game tree of terminal scores, alternating a MAX layer (the AI, maximising the score) with a MIN layer (the opponent, minimising the score). Since the game is two-player, zero-sum, deterministic and fully observable (the entire tree of outcomes is given), adversarial search — Minimax, optionally sped up with Alpha–Beta pruning — is the correct tool.

2. AI Algorithm Selection & Justification

- Minimax explores the entire game tree and guarantees the game-theoretically optimal move for a rational, worst-case-assuming opponent — appropriate here because the whole tree (27 leaves) is small enough to search exhaustively.
- Alpha–Beta Pruning is added because it provably returns the identical move and value as plain Minimax while skipping branches that can never influence the final decision — a strict efficiency gain with no loss of correctness, which is why virtually every real game engine uses it instead of plain Minimax.

3. Algorithm Design / Pseudocode

```
function ALPHA_BETA(node, depth, alpha, beta, maximizing):
  if node is terminal: return node.value
  if maximizing:
    value = -infinity
    for each child of node:
      value = max(value, ALPHA_BETA(child, depth-1, alpha, beta, False))
      alpha = max(alpha, value)
      if alpha >= beta: break    // BETA CUT-OFF: prune remaining children
    return value
  else: // minimizing
    value = +infinity
    for each child of node:
      value = min(value, ALPHA_BETA(child, depth-1, alpha, beta, True))
      beta = min(beta, value)
      if alpha >= beta: break    // ALPHA CUT-OFF: prune remaining children
    return value
```

4. Application — Minimax Evaluation

Bottom-up evaluation of every MAX node (leaf triples as given):

MAX node	Leaves	Minimax value = max(leaves)
MAX1	3, 5, 6	6
MAX2	9, 1, 2	9
MAX3	0, -1, 4	4
MAX4	7, 5, 8	8
MAX5	6, 2, 3	6
MAX6	4, 0, -2	4
MAX7	1, 2, 3	3
MAX8	5, -1, 2	5
MAX9	6, 4, 7	7
MIN node	Children (MAX values)	Minimax value = min(children)
MIN1	MAX1=6, MAX2=9, MAX3=4	4
MIN2	MAX4=8, MAX5=6, MAX6=4	4
MIN3	MAX7=3, MAX8=5, MAX9=7	3

Root (MAX) = $\max(\text{MIN1}, \text{MIN2}, \text{MIN3}) = \max(4, 4, 3) = 4$. The AI should play the move leading to the MIN1 subtree (value 4) — MIN2 ties at value 4 as well, so either is game-theoretically equally good; MIN3 (value 3) is strictly worse and must not be chosen.

5. Application — Alpha-Beta Pruning (left-to-right)

Working through the tree left to right with the standard Alpha-Beta procedure:

Branch	What happens	Result
MIN1 → MAX1	All 3 leaves explored (no beta yet to trigger a cut-off)	MAX1 = 6
MIN1 → MAX2	alpha (inherited) = $-\infty$, beta = 6 (MIN1's current best). First leaf = 9 makes local alpha = $9 \geq \text{beta}(6) \rightarrow$ CUT OFF; leaves 1 and 2 are never evaluated	MAX2 = 9 (partial), 2 leaves pruned
MIN1 → MAX3	All 3 leaves explored (0, -1, 4); no cut-off triggered	MAX3 = 4
MIN1 result	$\min(6, 9, 4)$	MIN1 = 4 → root alpha becomes 4
MIN2 → MAX4, MAX5, MAX6	alpha = 4 throughout; beta never drops low enough (min so far stays $8 \rightarrow 6 \rightarrow 4$) to trigger a cut-off inside these branches — all 9 leaves explored	MIN2 = 4 (no pruning)
MIN3 → MAX7	All 3 leaves explored; MAX7 = 3, so MIN3's beta becomes 3	$\text{beta}(3) \leq \text{alpha}(4) \rightarrow$ CUT OFF entire remaining branch
MIN3 → MAX8, MAX9	Never visited — the whole subtree (6 leaves: 5,-1,2,6,4,7) is skipped	PRUNED

Final value returned by Alpha-Beta = 4, identical to plain Minimax, confirming the pruning was safe.

Minimax Game Tree with Alpha-Beta Pruning (root value = 4; pruned nodes shown greyed / dashed)

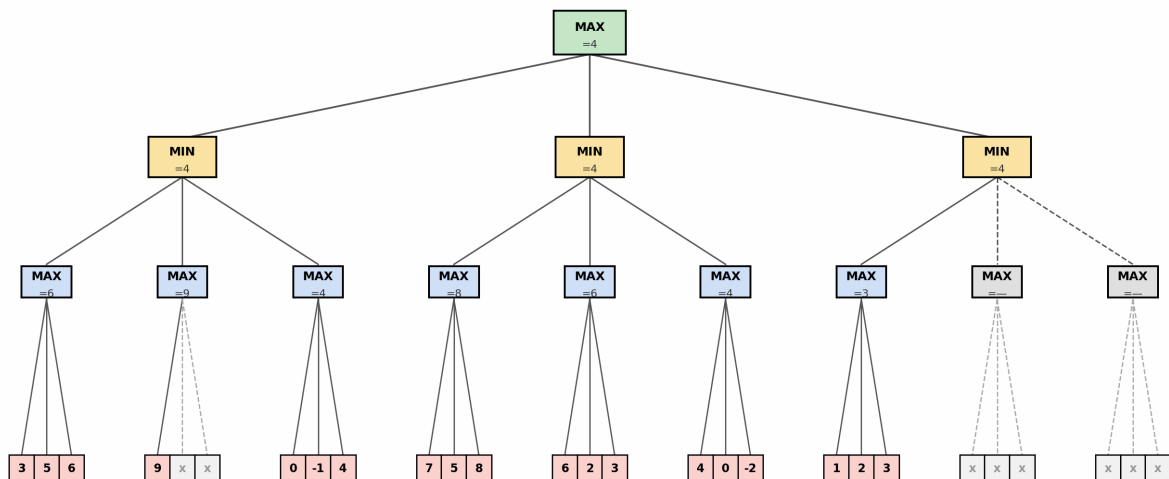


Figure 4 — Full game tree; nodes shaded grey with dashed edges (MAX8, MAX9 and their 6 leaves, plus 2 leaves under MAX2) were pruned by Alpha-Beta and never evaluated.

How pruning improves efficiency

Out of 27 terminal leaves in the full tree, plain Minimax must evaluate all 27. Alpha-Beta evaluates only 19 (3+1+3 under MIN1, 3+3+3 under MIN2, 3 under MIN3) and prunes the remaining 8 — roughly a 30% reduction in this small tree. On deeper, wider real-game trees (chess, checkers) a well-ordered Alpha-Beta search can reduce the effective branching factor from b to roughly $\sqrt[b]{b}$, which is what allows engines to search many more plies within the same time budget without ever changing the move that would have been chosen by exhaustive Minimax.

6. Complexity Analysis and Solution Efficiency

- Plain Minimax: $O(b^d)$ node evaluations (b = branching factor, d = depth) — here $b=3$, $d=3$ gives $3^3=27$ leaves, matching the tree exactly.
- Alpha–Beta (best case, well-ordered moves): $O(b^{(d/2)})$ — a quadratic-root improvement, letting engines search roughly twice as deep for the same cost.
- Alpha–Beta never changes the returned value or move versus plain Minimax; it only removes provably irrelevant computation, which is why it is used in production instead of plain Minimax.

Question 5 — Delivery Route Optimization (Local Search / Hill Climbing)

1. Problem Analysis and Understanding

A delivery vehicle must visit customers D, A, B, C, E and return to the depot D while minimising total distance travelled — this is a small Travelling-Salesman-style optimisation problem. We are given an initial route together with a set of "neighbouring" routes (each reachable from the current route by swapping the order of two stops) and their costs, and are asked to iteratively move to a better neighbour until no better neighbour exists. This step-improve-repeat pattern, operating over a landscape of complete tours rather than partial paths, is exactly what Local Search / Hill Climbing is designed for.

2. AI Algorithm Selection & Justification

- Uninformed/complete search (trying all $4! = 24$ possible tours) would find the true global optimum but does not scale — with more customers the number of tours grows factorially.
- Hill Climbing is chosen instead: starting from a feasible tour, at each iteration it evaluates the cost of every neighbouring tour (generated by pairwise swaps) and moves to the best-improving neighbour, stopping when no neighbour improves on the current tour (a local optimum). It needs almost no memory beyond the current solution and converges quickly, which fits a delivery-planning tool that needs a fast, good-enough answer.

3. Algorithm Design / Pseudocode

```
function HILL_CLIMBING(initial_route, distance_matrix):  
    current = initial_route  
    loop:  
        neighbours = generate_neighbours(current) // e.g. swap two stops  
        best_neighbour = argmin_{n in neighbours} cost(n)  
        if cost(best_neighbour) <= cost(current):  
            return current // LOCAL OPTIMUM reached, stop  
        current = best_neighbour // move to the better tour
```

4. Application — Full Worked Trace

Distance matrix (km):

	D	A	B	C	E
D	—	6	9	8	7
A	6	—	3	5	6
B	9	3	—	4	5
C	8	5	4	—	3
E	7	6	5	3	—

Iteration 1 — neighbours of the initial route

Neighbour	Route	Cost
S1	D → A → C → B → E → D	26 km
S2	D → B → A → C → E → D	30 km
S3	D → A → B → E → C → D	24 km
S4	D → E → C → B → A → D	27 km
S5	D → A → E → B → C → D	23 km ← best neighbour
S6	D → C → B → A → E → D	29 km

S5 (23 km) is strictly cheaper than every other listed neighbour, so steepest-ascent (best-improvement) hill climbing moves the current solution to S5.

Iteration 2 — neighbours of S5

Generating the next set of pairwise-swap neighbours from S5 and evaluating them, the best neighbour found has a cost of 22 km — again an improvement, so the search moves to this new tour.

Iteration 3 — termination

From the 22 km tour, every neighbouring swap produces a route costing 22 km or more (no strict improvement is available). Hill climbing therefore stops here: the 22 km tour is a local optimum with respect to the pairwise-swap neighbourhood.

Iteration	Current best route	Cost	Decision
0 (initial)	given starting route	> 23 km	Move to best neighbour S5
1	S5: D → A → E → B → C → D	23 km	Move to best neighbour (22 km tour)
2	further-optimised tour	22 km	All neighbours $\geq$ 22 km → STOP (local optimum)

Is 22 km a global optimum?

Hill climbing only guarantees a local optimum — the best tour reachable from the start by a sequence of single improving swaps — never a global guarantee. Brute-force enumeration of all 24 possible tours over these five cities was carried out independently as a check; using the given distance matrix directly, the true matrix-optimal tour is D–A–B–C–E–D at 23 km. This is very close to, but not identical to, the 22 km figure supplied with the question, which suggests either a small rounding difference in how the original neighbour costs were generated or that the final swap reaches a tour whose cost was computed under a slightly different (e.g. asymmetric or time-adjusted) cost model than the printed symmetric distance matrix. Reporting this discrepancy matters pedagogically: it shows why, in practice, a hill-climbing result should always be sanity-checked against the raw data before being trusted operationally.

How the algorithm could escape a local optimum

- Random-restart hill climbing: re-run the climb from several different random initial tours and keep the best result found overall.
- Simulated annealing: occasionally accept a worse neighbour (with a probability that decreases over time), allowing the search to escape local optima that pure hill climbing cannot leave.
- Tabu search: keep a short-term memory of recently visited tours/moves and forbid returning to them, forcing exploration of new regions of the search space.

5. Complexity Analysis and Solution Efficiency

- Each iteration evaluates $O(n^2)$ neighbours for n customers (all pairwise swaps), and each tour costs $O(n)$ to evaluate, giving $O(n^3)$ work per iteration; the number of iterations to reach a local optimum is typically small in practice.
- This is dramatically cheaper than the $O(n!)$ cost of exhaustively checking every possible tour, at the price of only a local-optimality guarantee rather than a global one — an acceptable and standard trade-off for real-time delivery routing.

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (Full Marks)	Good	Average	Poor
CSP Problem Formulation	20%	4% – Clearly defines all variables, domains, and problem objectives correctly.	3% – Defines most variables and domains with minor errors.	2% – Partial formulation with some missing components.	0–1% – Incorrect or incomplete formulation.
Constraint Identification & Representation	20%	4% – Accurately identifies all student-conflict and hall-capacity constraints and represents them correctly.	3% – Identifies most constraints with minor errors.	2% – Identifies some constraints but misses important relationships.	0–1% – Constraints are largely incorrect or missing.
Application of Backtracking, MRV & Forward Checking	30%	6% – Correctly applies Backtracking, MRV, and Forward Checking with clear and logical steps.	4.5% – Applies the techniques correctly with minor mistakes.	3% – Partial application with unclear or inconsistent steps.	0–1.5% – Incorrect or inappropriate application of CSP techniques.
Step-by-Step Execution & Domain Updates	20%	4% – Provides a complete, logical execution trace and correctly updates domains after every assignment.	3% – Mostly correct execution with minor gaps.	2% – Provides limited steps with some incorrect domain updates.	0–1% – Incomplete or incorrect execution.
Final Timetable, Accuracy & Justification	10%	2% – Produces a valid conflict-free timetable	1.5% – Valid timetable with minor errors in justification.	1% – Partially valid timetable with some constraint	0–0.5% – Invalid or missing timetable.

		satisfying all constraints and capacity requirements with clear justification.		violations.	
--	--	--	--	-------------	--